

Paginacja, metody ochrony przed botami

Wykłady 10

Bartosz Lauks

12 maja 2026



① Paginacja

② reCAPTCHA

① Paginacja

② reCAPTCHA

Problem rosnącej ilości danych w aplikacjach webowych

- Rozwój każdej aplikacji internetowej zwykle zaczyna się od niewielkiej ilości danych kilku produktów, użytkowników czy wpisów.
- W początkowej fazie projektowania systemu często wystarcza proste wyświetlanie zawartości w jednym widoku, bez dodatkowej logiki ograniczającej ilość prezentowanych informacji.
- Z czasem jednak aplikacja zyskuje na popularności, baza danych rośnie, a liczba rekordów zaczyna przekraczać setki, a nawet tysiące. W tym momencie pojawia się pierwszy poważny problem
- Jak wydajnie i przejrzysto zaprezentować użytkownikowi dużą ilość danych?

Paginacja

Implementacja paginacji do aplikacji z wykładu

- W ostatnim wykładzie aplikacja została rozszerzona o funkcjonalność postów oraz powiązanych z nimi komentarzy.
- Obecnie podstrona odpowiedzialna za wyświetlanie postu i przypisanych do niego komentarzy nie jest odporna na problem rosnącej ilości danych.
- W przypadku dużego zaangażowania i dodania do wątku ponad tysiąca komentarzy, aplikacja będzie każdorazowo pobierać i wyświetlać wszystkie rekordy z bazy, co może doprowadzić do przeciążenia. Wąskim gardłem staje się wówczas połączenie aplikacji z bazą danych. Nie wspominając już o potencjalnie wysokich kosztach takiego rozwiązania.
- Warto zauważyć, że większość użytkowników serwisu będzie zainteresowana jedynie najnowszymi komentarzami, co stanowi dodatkowy argument za tym, aby nie wyświetlać całego zbioru danych naraz.

Struktura paginacji (komponenty)

- **Frontend (HTML + JavaScript)** – wyświetla dane oraz przyciski do przechodzenia między stronami.
- **Backend (PHP + PDO)** – obsługuje zapytania do bazy danych, pobiera tylko określoną liczbę rekordów i przekazuje je do frontu.
- **Baza danych (np. MySQL)** – przechowuje dane, np. listę użytkowników, produktów, artykułów itd.

Paginacja

Paginacja statyczna - Jak działa krok po kroku?

- Aby zapewnić skalowalność, należy zmodyfikować podstronę wyświetlającą komentarze w taki sposób, aby na podstawie przesłanych danych wejściowych możliwe było określenie, która strona z danymi ma zostać wyświetlona.
- Do tego celu idealnie nadają się **parametry zapytania (query parameters)**, które mogą przekazywać do skryptu informację o numerze bieżącej strony, np. **page=2**.
- Parametry zostaną pobrane za pomocą **super-globalnej `$_GET`**

SQL - LIMIT & OFFSET

- Ważnym będzie ustalenie dwóch istotnych danych. Pierwszym będzie maksymalna ilość wyświetlanych komentarzy na stronie. Daną tę aktualnie ustawimy na sztywną wartość **5**.
- W zapytaniu SQL do bazy danych możemy wykorzystać słowo kluczowe **LIMIT**, którego parametr określa ile maksymalnie wierszy zostanie zwróconych.

Listing 1: Zapytanie SQL z LIMIT

```
1 SELECT * FROM comments LIMIT 5;  
2 // wróci pierwsze 5 rekordów z tabeli comments
```

SQL - LIMIT & OFFSET cd.

- Niestety, zastosowanie samego parametru **LIMIT** nie rozwiązuje problemu stronicowania wyników.
- Brakuje określenia, od którego rekordu silnik bazodanowy ma rozpocząć zwracanie danych, a to właśnie umożliwia wyświetlanie kolejnych stron.
- Taką funkcjonalność zapewnia parametr **OFFSET**.
- Wystarczy, aby skrypt PHP za każdym razem pobierał numer strony z **parametru zapytania (query parameter)** i mnożył go przez maksymalną liczbę wyników wyświetlanych na jednej stronie.
- W ten sposób uzyskamy w pełni działające stronicowanie wyników.

Paginacja

Wzór na OFFSET

- Na podstawie numeru strony przekazywanego przez przeglądarkę użytkownika oraz parametru **LIMIT** można obliczyć, które rekordy powinny zostać pobrane z bazy danych.
- Dzięki temu prostemu wzorowi można stworzyć uniwersalny mechanizm stronicowania wyników.

Wzór

$$OFFSET = (page - 1) \times limit$$

- **page** = numer strony
- **limit** = liczba rekordów na stronę

Paginacja

SQL - LIMIT & OFFSET cd.

Listing 2: Zapytanie SQL z LIMIT & OFFSET

```
1 SELECT * FROM comments LIMIT 5 OFFSET 10;
2 // Zwróci 5 rekordów, począwszy od 11.
3
4 SELECT * FROM comments LIMIT 10 OFFSET 20;
5 // Przykład: dla strony 3 i 10 wyników na strone:
6 // pomin pierwsze 20 rekordów (OFFSET 20),
7 // pokaz kolejne 10 (LIMIT 10).
8 LIMIT 20, 10 // Skrócony zapis
```

<localhost/lecture10/postPaginationV1.php?id=1&page=1>

Paginacja

Ulepszenie paginacji

- Obecna wersja paginacji nie zwraca informacji o maksymalnym stronicowaniu.
- Powoduje to, że mimo braku wyników użytkownik nadal może przechodzić między stronami, które nie zawierają żadnych danych.
- Rozwiązaniem tego problemu jest wykonanie dodatkowego zapytania do bazy danych, które zwróci całkowitą liczbę dostępnych rekordów.
- Na tej podstawie można obliczyć maksymalną liczbę stron przy zadanym limicie wyników na stronę.
- Dzięki tym danym można dodatkowo ukryć przycisk przejścia do następnej strony w przypadku, gdy użytkownik znajduje się na ostatniej stronie, a także dodać numerację stron.

<localhost/lecture10/postPaginationV2.php?id=1&page=1>

Paginacja

Walidacja danych w systemie paginacji

- Implementując system paginacji w aplikacji webowej, nie można zapomnieć o bezpieczeństwie, ponieważ przez nieuwagę możemy stworzyć nowe wektory ataku.
- Parametry paginacji (np. **page**, **limit**) pochodzą od użytkownika.
- Mogą być zmodyfikowane ręcznie w **URL**.
- Brak walidacji może prowadzić do:
 - błędów aplikacji,
 - przeciążenia bazy danych,
 - podatności bezpieczeństwa.

Walidacja danych w systemie paginacji

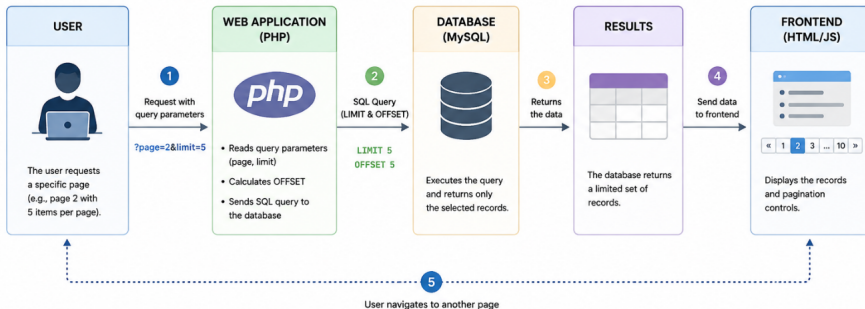
- Parametry wymagające kontroli
 - **page** - numer strony
 - **limit** - liczba rekordów na stronę
- Oba parametry muszą być:
 - liczbami całkowitymi
 - większe od zera
 - dopuszczalnym zakresie.

Ulepszenie paginacji. Walidacja i dowolny zmienny limit

- Zastosujmy dobre praktyki. Obecnie limit wyników wyświetlanych na stronie jest domyślnie ustawiony na **5**.
- Jeśli chcemy, aby nasza aplikacja była bardziej elastyczna, powinniśmy umożliwić użytkownikowi samodzielne dostosowanie tego parametru.
- Niestety, jeśli damy użytkownikowi pełną dowolność, może on ustawić limit na wartość powyżej tysiąca lub jeszcze wyższą.
- W praktyce umożliwia to całkowite ominięcie systemu stronicowania.
- Lepszym rozwiązaniem będzie udostępnienie ograniczonego zakresu wartości limitu, na przykład kilku stałych opcji: 5, 10 lub 20 wyników na stronę.

<localhost/lecture10/postPaginationV3.php?id=1&page=1>

How Pagination Works



EXAMPLE SQL QUERY

```
SELECT * FROM comments
WHERE post_id = 1
ORDER BY created_at DESC
LIMIT 5 OFFSET 5;
```

- Select comments for a specific post
- Order by newest first
- Get 5 records
- Skip the first 5 records (page 2)

PAGINATION CONCEPT

If limit = 5:

Page 1	Page 2	Page 3	...	Page N
OFFSET 0	OFFSET 5	OFFSET 10		OFFSET (N-1) × 5

i $OFFSET = (page - 1) \times limit$

Wady LIMIT i OFFSET Problemy wydajnościowe

- OFFSET przy bardzo dużych tabelach może działać wolno.
- Problem polega na działaniu silników bazodanowych.
- Baza danych nie „przeskakuje” od razu do wybranego miejsca.
- Aby zwrócić dane, musi najpierw odczytać i pominąć wcześniejsze rekordy.
- Dlatego w dużych systemach stosuje się często:
 - **cursor pagination,**
 - **keyset pagination,**
 - **lazy loading.**

Paginacja cursor-keyset

- Jest to strategia, która zamiast numerów stron wykorzystuje wskaźnik (**cursor**). Przykład adresu:
</app/posts?cursor=105>
- Parametr [cursor=105](#) wskazuje ostatni rekord z poprzedniej strony, czyli miejsce, od którego należy kontynuować pobieranie danych.
- Informacja ta jest następnie wykorzystywana w technice **Keyset Pagination**.
- Strategia ta polega na pobieraniu danych za pomocą warunków **WHERE** opartych o klucze sortujące, np.:
WHERE id > 105
- Dzięki temu baza danych nie musi pomijać wcześniejszych rekordów, co znacząco zwiększa wydajność przy dużych tabelach.

Paginacja

Przykład strategii cursor-keyset

Listing 3: Zapytanie SQL według strategii cursor-keyset

```
1 SELECT * FROM comments
2 ORDER BY id ASC
3 LIMIT 10;
4 // Pierwsze pobranie danych. Pobierz pierwsze 10 rekordów,
5 // Krok 2
6 SELECT * FROM comments
7 WHERE id > 10
8 ORDER BY id ASC
9 LIMIT 10;
10 //Pobierz kolejne 10 rekordów. Pobierz rekordy o id większym niż 10,
11 //LIMIT 10 określa liczbę wyników.
```

Wady cursor-keyset

- **Brak skakania po stronach** (np. strona 10)
- **Trudna numeracja stron w UI** (1,2,3,4 itd.)
- **Większa złożoność implementacji backendu**
- **Trudne cofanie się** (previous page)
- **Wrażliwość na zmiany danych w trakcie paginacji**
- **Wymaga stabilnego sortowania** (np. id + created_at)
- **Trudniejsze debugowanie niż OFFSET**

Paginacja

Paginacja Infinite scroll (lazy loading)

- Ostatnim typem paginacji będzie tak zwany "**Infinite scroll**" znany z takich stron WWW jak Facebook, Twitter (X), Instagram itd.
- Czyli dynamiczne ładowanie treści podczas przewijania strony przez użytkownika.
- Zaletą tego podejścia płynne i ciągłe przeglądanie treści przez użytkownika, nie musi klikać w kolejne strony nowe treści ładują się automatycznie podczas przewijania.
- Aplikacja ładuje początkowo tylko ograniczoną liczbę elementów, co skraca czas wczytywania strony.
- Wydajne wykorzystanie zasobów klienta jak i serwera.

Paginacja

Paginacja Infinite scroll - sposób implementacji

- Podejście to wymaga rozdzielenia procesu generowania danych od ich wyświetlania na dwie oddzielne podstrony.
- Jest to konieczne, aby zachować separację między logiką generowania a prezentacją danych.
- Pierwsza podstrona będzie odpowiedzialna wyłącznie za zwracanie treści (np. komentarza) na podstawie parametrów przekazanych w adresie URL.
- Druga podstrona będzie asynchronicznie wysyłać zapytania do pierwszej, aby pobrać dane i je wyświetlić.
- Taki sposób pozyskiwania i prezentowania danych jest zbliżony do architektury **REST API**.

<localhost/lecture10/postPaginationV5.php?id=1>

① Paginacja

② reCAPTCHA

Problemy bezpieczeństwa aplikacji WWW

- Współczesne aplikacje webowe, choć niezwykle funkcjonalne i dostępne z każdego miejsca na świecie, są również narażone na szereg zagrożeń.
- Otwartość internetu sprawia, że każdy formularz, interfejs logowania czy API może stać się celem ataku.
- Wraz z rozwojem technologii, coraz częściej nie są to ataki wykonywane ręcznie przez pojedynczych użytkowników, lecz działania zautomatyzowane, wykonywane przez boty, czyli programy symulujące aktywność człowieka.

Problemy bezpieczeństwa aplikacji WWW a boty

- Najczęstsze problemy bezpieczeństwa aplikacji webowych związane z botami to:
 - **Spam** - Boty rozsyłają reklamy, linki do złośliwych stron lub sztucznie generują treści, które zanieczyszczają dane w systemie i obniżają jego wiarygodność.
 - **Automatyczna rejestracja i logowanie** - Boty potrafią masowo zakładać konta użytkowników lub wykonywać tysiące prób logowania (tzw. ataki brute force), próbując odgadnąć hasła.
 - **Obciążanie serwera (Denial of Service)** - Zautomatyzowane systemy mogą celowo wysyłać setki lub tysiące żądań do serwera w krótkim czasie, próbując doprowadzić do jego przeciążenia, a nawet chwilowego wyłączenia usługi dla prawdziwych użytkowników.

reCAPTCHA

Czym jest reCAPTCHA?

- **reCAPTCHA** to usługa oferowana przez Google, która pozwala odróżnić prawdziwych użytkowników (ludzi) od zautomatyzowanych programów (botów).
- Jej głównym celem jest zabezpieczenie formularzy, systemów logowania, komentarzy, rejestracji i innych funkcji przed spamem oraz atakami typu brute force.
- Narzędzie zostało uruchomione w 2007 roku i udostępnione do bezpłatnej implementacji w aplikacjach WWW.

Wersje reCAPTCHA

- **reCAPTCHA v1 (już nieaktywna)** - Użytkownik musiał przepisać zniekształcony tekst (captcha) z obrazka. Była mało przyjazna i często trudna do odczytania.
- **reCAPTCHA v2 „Nie jestem robotem”** - Użytkownik zaznacza checkbox „Nie jestem robotem”. Czasem musi dodatkowo rozwiązać zadanie. (np. wybrać zdjęcia z przejściami dla pieszych, autobusami, znakami drogowymi itp.).
- **reCAPTCHA v3 całkowicie niewidoczna** - Nie wymaga interakcji ze strony użytkownika. System analizuje zachowanie użytkownika na stronie (np. ruchy myszki, czas spędzony na stronie, sposób przewijania).

reCAPTCHA

reCAPTCHA a prywatność

- reCAPTCHA analizuje dane użytkownika (adres IP, cookies, sposób poruszania się po stronie).
- Użytkownik korzystając z formularza z reCAPTCHA, akceptuje zasady prywatności Google.
- reCAPTCHA v3 wzbudza pewne kontrowersje, bo działa w tle i może gromadzić dane bez wyraźnej zgody.

reCAPTCHA

Integracja reCAPTCHA

Aby użyć reCAPTCHA na stronie:

- Rejestrujesz swoją domenę na stronie <https://www.google.com/recaptcha>.
- Wybierasz wersję (v2 lub v3).
- Otrzymujesz **klucz publiczny (site key)** i **klucz prywatny (secret key)**.
- Wklejasz odpowiedni kod JavaScript oraz fragment HTML na stronie.
- Na w PHP weryfikujesz token reCAPTCHA za pomocą zapytania do API Google.

<localhost/lecture10/reCAPTCHA/>

reCAPTCHA

Co się dzieje po stronie serwera? (v2)

- Po zaznaczeniu reCAPTCHA (v2), użytkownik otrzymuje token (ciąg znaków).
- Token jest wysyłany razem z formularzem do Twojego serwera.
- PHP łączy się z API Google i sprawdza token, przekazując:
 - Secret key.
 - token od użytkownika.
 - adres IP (opcjonalnie)
- Google zwraca informację, czy użytkownik został uznany za człowieka (**success: true**), lub nie.

reCAPTCHA

Co się dzieje po stronie serwera? (v3)

- **reCAPTCHA v3**, w przeciwieństwie do poprzedniej wersji, nie zwraca jednoznacznej odpowiedzi, czy użytkownik korzystający z serwisu jest botem.
- Google analizuje zachowanie użytkownika i zwraca wynik punktowy (**score**) od 0.0 do 1.0.
- Im wyższy wynik (np. **0.9**), tym większe prawdopodobieństwo, że użytkownik jest człowiekiem.
- Aplikacja sama decyduje, jaki próg uznać za bezpieczny (np. **score > 0.5**).
- Ostateczna decyzja o akceptacji lub odrzuceniu użytkownika należy więc do dewelopera aplikacji.

Dziękuję za uwagę !

Bartosz Lauks